Shell Scripting

What is shell ?

- Shell is the interface between user and kernal
- When a user execute commands the shell will take and verify if the syntax is valid then shell will give commands instructions to kernal
- Kernal will interact with hardware and it will perform the operations
- Shell is the responsible to read commands given by the user

- In the linux kernal is the heart of the operating system which will interact with hardware
- Shell is the mediator between user and kernal .

#Types of shell:

There are several shells available in linux

1. Bourne shell
2. Bash shell
3. Korn shell
4. C shell
5. T shell
6. Z shell

Out of these all shells , bash is the most powerful shell , and it is most commonly used shell by admin/users.

cat /etc/shells - **It displays all the shells which are available in the linux**

To check for default shell which is present in linux – echo $SHELL

What is scripting ?

- Sequence of commands saved in a file is called scripting
- The file which contains sequence of commands is called script file.

What is Shell Scripting : the process of writing the sequence of commands in a file and executing the file using shell is called as shell-scripting

- Script file will have .sh extension
- We will use shell scripting to automate some daily tasks ex: create files, delete files, backup , start and stop servers

Real time use cases for shell scripting

1. Creating / deleting log files
2. Taking log files backup
3. Start & stop the servers
4. Installing softwares
5. Daily server restart .

Write and execute first shell script in linux?

- Create a file with a name for ex: task.sh
- Edit the file using vi editor and write the script
- Add execute permissions for the file using "chmod command"
- Execute the file using this command " bash task.sh"

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat task.sh
#!/bin/bash



ls
pwd
#!/bin/bash



ls
pwd

whoami
ls -l
uname
uname -r
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash task.sh
dltuser.sh  hello.sh  shell.sh  task.sh  user.sh  user1.sh
/home/ubuntu/shell-scripting
dltuser.sh  hello.sh  shell.sh  task.sh  user.sh  user1.sh
/home/ubuntu/shell-scripting
ubuntu
total 24
-rwxrwxr-x 1 ubuntu ubuntu  236 Jul 16 12:34 dltuser.sh
-rwxrwxr-x 1 ubuntu ubuntu 2221 Jul 16 12:34 hello.sh
-rwxrwxr-x 1 ubuntu ubuntu  824 Jul 16 12:34 shell.sh
-rwxrwxr-x 1 ubuntu ubuntu   73 Jul 16 12:35 task.sh
-rwxrwxr-x 1 ubuntu ubuntu  963 Jul 16 12:34 user.sh
-rwxrwxr-x 1 ubuntu ubuntu  392 Jul 16 12:34 user1.sh
Linux
6.8.0-1010-aws
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

#!interpretor – which tells what shell we need to use , this we called as shebang line

- It is highly recommended to use shebang line at the starting line of the script .
- Here iam specifying she-bang to use bash shell to execute my script.

#task2:

Vi welcome.sh

Echo “Enter your name”

read name

echo “Hey $name , Good Morning..!!”

echo is used to print the statement

read is used to to take the input from user

```
ubuntu@ip-172-31-37-26:~/shell-scripting$
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat welcome.sh
#!/bin/bash


echo " Enter your name: "
read name
echo "Hi $name , Good Morning ..!"

ubuntu@ip-172-31-37-26:~/shell-scripting$ bash welcome.sh
 Enter your name:
vaseem
Hi vaseem , Good Morning ..!
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Note: In Ubuntu “bash “ is a default shell

- Some other distributions may have (or) may have not default shell
- So by using shebang we tell which shell to use.

I want to print date and storage of disk and ram , also I want to extract columns and want to fetch only disk size & ram size by using awk.

Vi test.sh

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat test.sh
echo " Today date is $(date)"
echo " Disk space is $(df -h) "
echo " ram : $(free -h)"
echo "disk is $(df -h | awk 'NR==2 {print $2}') & ram is $(free -h | awk 'NR==2 {print $2}')"

ubuntu@ip-172-31-37-26:~/shell-scripting$ bash test.sh
 Today date is Tue Jul 16 13:02:20 UTC 2024
 Disk space is Filesystem      Size  Used Avail Use% Mounted on
/dev/root       6.8G  2.6G  4.2G  39% /
tmpfs           479M     0  479M   0% /dev/shm
tmpfs           192M  880K  191M   1% /run
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/xvda16     881M  133M  687M  17% /boot
/dev/xvda15     105M  6.1M   99M   6% /boot/efi
tmpfs            96M   12K   96M   1% /run/user/1000
 ram :               total        used        free      shared  buff/cache   available
Mem:           957Mi       392Mi       252Mi       892Ki       472Mi       564Mi
Swap:            0B          0B          0B
disk is 6.8G & ram is 957Mi
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

| - pipe is used to take output of first command to the input for the second command.

Variables: variables are used to store the data

- Vary means change , anyone can change the variables.
- We can say variables are placeholders to hold the data
- Variable will represent data in key-value format
- In shell scripting there is no concept of data-type so every value will be treated as text only.

if you want to use command inside the echo you should use parenthesis (command)

Ex : name=ashok , age=24 , married=yes

- Variables are divided into two types
  1. Environment variables : it will be used by o.s , we call these variables as system variables
     We can see all the system variables using "ENV " Command.

  2. User defined variables : Based on pur requirement we will define the variables those are called as user defines variables .
     For ex: hero=dulquer , age=35

Vi num.sh

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ vi num.sh
ubuntu@ip-172-31-37-26:~/shell-scripting$ ubuntu@ip-172-31-37-26:~/shell-scripting$ cat num.sh
a=10
b=20
echo "sum is:, $a+$b"
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash num.sh
sum is:, 10+20
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Rules to define variables:

- It is recommended to use upper case for variable names.
- If variable contains multiple words then we can use underscore(_) to separate the words
- Variables should not start with digit
- We shouldn't use special symols like - , #,@

Command line arguments:

- The arguments which we will pass while executing the script files are called as Command line arguments

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat linux.sh
# Check if the script has received the expected number of arguments
if [ $# -ne 2 ]; then
        echo "Usage: $0 <arg1> <arg2> ..."
        exit 1
fi
echo "Total number of arguments: $#"
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"


echo "All arguments:"
for arg in "$@"; do
            echo "$arg"
done
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash linux.sh arg1 arg2
Total number of arguments: 2
Script name: linux.sh
First argument: arg1
Second argument: arg2
All arguments:
arg1
arg2
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

$# - represent no of arguments

$0 – it displays script file

$1 – first argument

$2 – second arguments

Note: At the runtime if you want to send some input to the script file we go for command line arguments.

Comments:

- Comments are used to provide metadata about our script.
- Metadata means data about data
- By looking at comments others can easily understand our script
- To write single line comment we use “#”

#for multiline comments we use <<delimeter ---------- inside this keep data

delimeter

Conditional Statements:

- If we want to execute our script commands based on some conditions then we can use control statements

Syntax :

If -else

Syntax	example

if [condition]; then
echo statement
else
echo statemet
fi
	Vi age.sh
	echo “Enter your age”
	read age
	if [$age –ge 18]; then
	echo “Your eligible”
	else
	echo “Your not eligible for vote”
	fi

If-elif : If we want to check more than one condition then we can use if-elif

syntax	**example**

num=10
if [condition]; then
echo “statement”
elif [condition2]; then
echo “statement”
else
echo “statement”
fi
	num=10
	if [$num -gt 0]; then
	echo "$num is positive"
	elif [$num -lt 0]; then
	echo "$num is negative"
	else
	echo "$num is zero"
	fi

Case : if we want to check several conditions then we will go for case statement

Syntax	example
case $variable in pattern1) echo “statement” ;; Pattern2) Echo “statement” ;; Pattern3) Echo “statement” ;; *) Echo “statement” ;; esac	day=$(date +%u) # Get the day of the week as a number (1-7) case $day in 1) echo "Today is Monday" ;; 2) echo "Today is Tuesday" ;; 3) echo "Today is Wednesday" ;; 4) echo "Today is Thursday" ;; 5) echo "Today is Friday" ;; 6|7) echo "It's the weekend!" ;; *) echo "Invalid day" ;; esac

Iterative statements:

iterative statements are used to repeat a block of code multiple times, typically based on a condition or a set of values

1. For Loop:

The `for` loop iterates over a list of items. Here's the syntax:

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat items.sh
for item in item1 item2 item3; do
            echo "Current item is: $item"
    done
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash items.sh
Current item is: item1
Current item is: item2
Current item is: item3
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Syntax:	Example
For ((initialize ; condition ; increment/decrement)) Do Statement done	For ((i=1; i<=10; i++)) Do echo "$i" done

- Loops are used to execute cond (or) statements repeatedly
- It is going to check the condition i<=10 or not , if it is less than it will print & iterates I++
- It is going to execute the statements repeatedly

Task : I want to create a user using shell script

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat user1.sh
for (( i=1; i<=4; i++ )); do
        read -p "Enter Username: " username
        read -sp "Enter Password: " password
        echo
        encrypted_password=$(openssl passwd -6 "$password")
        sudo useradd -m -p "$encrypted_password" "$username"
        if [ $? -eq 0 ]; then
                        echo "User '$username' created successfully."
                            else
                                        echo "Failed to create user '$username'."
                                            fi
                                    done
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash user1.sh
Enter Username: vaseem
Enter Password:
User 'vaseem' created successfully.
```

While loop:

- For loop is for range based loop , if I know the range then I use for , if not I go for while loop

syntax	example
While [condition] Do Echo ‘statement’ Done	count=5 while [$count -gt 0] do echo "Countdown: $count" count=$((count - 1)) done echo "Liftoff!"

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat greater.sh
i=10
while [ $i -ge 0 ]
do
    echo "Reverse order $i "
     let i--
done
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash greater.sh
Reverse order 10
Reverse order 9
Reverse order 8
Reverse order 7
Reverse order 6
Reverse order 5
Reverse order 4
Reverse order 3
Reverse order 2
Reverse order 1
Reverse order 0
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Note: we should never write infinite loops n the script , always we need to write condition to break the loop execution based on our requirement

Until loop : Until loop stop when the condition is True .

While loop stops when the condition is false

Ex:

```
Count=10
I=10
Until [ $i –gt $count ] ;
Do
    Echo “$i”
     Let i++
Done
```

Functions: **Functions are used to perform some action , with the help of functions overall functionality of a function can be divided into smaller or logical parts , which can be called to perform their task.**

- **It helps us to check our programs part by part , we can resue the function wherever we want .**

Example:

#defining function

```
Function welcome() {
    Echo “Welcome to $1”
}
Welcome  # calling function
```

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ cat welcome.sh
# defining function


function welcome() {
        echo "Welcome to, $1"
}

read -p "Enter your name: " username
welcome $username # calling function
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash welcome.sh
Enter your name: DevOps
Welcome to, DevOps
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Task : I want to create and delete the users using shell script and also using case statement if I give input c it has to create the user , if input id d it has to delete the user

```
ubuntu@ip-172-31-37-26:~/shell-scripting$  cat user.sh
#!/bin/bash

<< comment

Inside this we can write comment lines

comment




#function create_user {
#function defining


        read -p "Enter Username:" username
        read -p "Enter Password:" password
        sudo useradd -m -p "$password" "$username"

        echo "Created successfully"
}
#for (( i=1 ; i<=5; i++ ))
#do
#create_user
#done

#create_user #function calling



function delete_user {
                read -p "Enter the user name to be deleted: " username

                        sudo userdel -r $username

                                echo "USER $username deleted successfully"
                        }

#                       for (( i=1 ; i<=4 ; i++ ))
#                       do
#                               delete_user
#                               done
# i have a shell script

function show_usage {
        echo "Welcome to user Management"
        echo "To create user use c as argument"
        echo "To delete user use d as argument"


}
case "$1" in
        c)
        echo " Create user in Progress"
        create_user
        ;;
        d)
        echo "Delete user in progress"
        delete_user
        ;;
        *)
        show_usage
        ;;

esac
```

```
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash user.sh
Welcome to user Management
To create user use c as argument
To delete user use d as argument
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash user.sh c
 Create user in Progress
Enter Username:vaseem
Enter Password:vaseem
useradd: user 'vaseem' already exists
Created successfully
ubuntu@ip-172-31-37-26:~/shell-scripting$ bash user.sh d
Delete user in progress
Enter the user name to be deleted: vaseem
userdel: vaseem mail spool (/var/mail/vaseem) not found
USER vaseem deleted successfully
ubuntu@ip-172-31-37-26:~/shell-scripting$
```

Day 9:

Task : task is to create a bash script that takes a directory path as a command-line argument and performs a backup of the directory. The script should create timestamped backup folders and copy all the files from the specified directory into the backup folder.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat backup.sh
#!/bin/bash


<< comment
This script takesbackup of given directory
usage:
./backup.sh <path of directory> <path of backup dir>
comment

source_dir="$1"
target_dir="$2"
timestamp=$(date '+%Y-%m-%d-%H-%M-%S')
backup_dir="${target_dir}/backup_${timestamp}"

function create_backup {

zip -r "${backup_dir}.zip" "${source_dir}" > /dev/null

if [ $? -eq 0 ]; then

        echo "Backup Created Successfully"
else
        echo "Backup was not created for $timestamp"
fi

}
create_backup
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash backup.sh /home/ubuntu/shell-scripting/scripts/ /home/ubuntu/backups/
Backup Created Successfully
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

Additionally, the script should implement a rotation mechanism to keep only the last 3 backups. This means that if there are more than 3 backup folders, the oldest backup folders should be removed to ensure only the most recent backups are retained.

The script will create a timestamped backup folder inside the specified directory and copy all the files into it. It will also check for existing backup folders and remove the oldest backups to keep only the last 3 backups.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat backup.sh
#!/bin/bash


<< comment
This script takesbackup of given directory
usage:
./backup.sh <path of directory> <path of backup dir>
comment

source_dir="$1"
target_dir="$2"
timestamp=$(date '+%Y-%m-%d-%H-%M-%S')
backup_dir="${target_dir}/backup_${timestamp}"

function create_backup {

zip -r "${backup_dir}.zip" "${source_dir}" > /dev/null

if [ $? -eq 0 ]; then

        echo "Backup Created Successfully"
else
        echo "Backup was not created for $timestamp"
fi

}

function perform_rotation {
        backups=($(ls -t "$target_dir/backup_"*.zip))
        echo "${backups[@]}"

        if [ "${#backups[@]}" -gt 3 ]; then
                backups_to_remove=("${backups[@]:3}")
                for backup in "${backups_to_remove[@]}";
                do
                        rm "$backup"
                done

           fi

        }

create_backup
perform_rotation
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash backup.sh /home/ubuntu/shell-scripting/scripts/ /home/ubuntu/backups/
Backup Created Successfully
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash backup.sh /home/ubuntu/shell-scripting/scripts/ /home/ubuntu/backups/
Backup Created Successfully
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash backup.sh /home/ubuntu/shell-scripting/scripts/ /home/ubuntu/backups/
Backup Created Successfully
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cd /home/ubuntu/backups/
ubuntu@ip-172-31-37-26:~/backups$ ls
backup_2024-07-18-11-59-18.zip  backup_2024-07-18-11-59-20.zip  backup_2024-07-18-11-59-21.zip
ubuntu@ip-172-31-37-26:~/backups$
```

I want to run complete automation without executing the command , it is like alaram wise for that we use crontab

Cron – tool allows scheduling the jobs

In linux there is a tool called Crontab which list cronjobs

Job nothing but I want schedule a job at particular time for ex: alaram in our phone is a cronjob

Crontab –e

Schedule the time

0 20 * * * bash <path of script> <src_dir> <target_dir>

Day 10 : Write a Bash script that automates the process of analyzing log files and generating a daily summary report. The script should perform the following steps:

1. **Input:** The script should take the path to the log file as a command-line argument.
2. **Error Count:** Analyze the log file and count the number of error messages. An error message can be identified by a specific keyword (e.g., "ERROR" or "Failed"). Print the total error count.
3. **Critical Events:** Search for lines containing the keyword "CRITICAL" and print those lines along with the line number.
4. **Top Error Messages:** Identify the top 5 most common error messages and display them along with their occurrence count.
5. **Summary Report:** Generate a summary report in a separate text file. The report should include:
    - Date of analysis
    - Log file name
    - Total lines processed
    - Total error count
    - Top 5 error messages with their occurrence count
    - List of critical events with line numbers

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat log.sh
usage() {
        echo "Usage: $0 <path_to_log_file>"
        exit 1
}

if [ $# -ne 1 ]; then
            usage
fi

LOG_FILE=$1

if [ ! -f "$LOG_FILE" ]; then
            echo "Error: Log file $LOG_FILE does not exist."
            exit 1
fi

ERROR_KEYWORD="ERROR"
CRITICAL_KEYWORD="CRITICAL"
DATE=$(date +"%Y-%m-%d")
SUMMARY_REPORT="summary_report_$DATE.txt"
ARCHIVE_DIR="processed_logs"

{
    echo "Date of analysis: $DATE"
    echo "Log file name: $LOG_FILE"
} > "$SUMMARY_REPORT"

TOTAL_LINES=$(wc -l < "$LOG_FILE")
echo "Total lines processed: $TOTAL_LINES" >> "$SUMMARY_REPORT"

ERROR_COUNT=$(grep -c "$ERROR_KEYWORD" "$LOG_FILE")
echo "Total error count: $ERROR_COUNT" >> "$SUMMARY_REPORT"

echo "List of critical events with line numbers:" >> "$SUMMARY_REPORT"
grep -n "$CRITICAL_KEYWORD" "$LOG_FILE" >> "$SUMMARY_REPORT"

declare -A error_messages
while IFS= read -r line; do
            if [[ "$line" == *"$ERROR_KEYWORD"* ]]; then
                        message=$(echo "$line" | awk -F"$ERROR_KEYWORD" '{print $2}')
                                ((error_messages["$message"]++))
           fi
done < "$LOG_FILE"

echo "Top 5 error messages with their occurrence count:" >> "$SUMMARY_REPORT"
for message in "${!error_messages[@]}"; do
      echo "${error_messages[$message]} $message"
      done | sort -rn | head -n 5 >> "$SUMMARY_REPORT"


      if [ ! -d "$ARCHIVE_DIR" ]; then
        mkdir -p "$ARCHIVE_DIR"
fi

cp "$LOG_FILE" "$ARCHIVE_DIR/"
echo "Log file has been moved to $ARCHIVE_DIR."
```

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash log.sh /home/ubuntu/day10.log
Log file has been moved to processed_logs.
Date of analysis: 2024-07-18
Log file name: /home/ubuntu/day10.log
Total lines processed: 2002
Total error count: 13
List of critical events with line numbers:
Top 5 error messages with their occurrence count:
1  [LearnerHandler-/10.10.34.13:57617:LearnerHandler@562] - Unexpected exception causing shutdown while sock still open
1  [LearnerHandler-/10.10.34.13:57438:LearnerHandler@562] - Unexpected exception causing shutdown while sock still open
1  [LearnerHandler-/10.10.34.13:57354:LearnerHandler@562] - Unexpected exception causing shutdown while sock still open
1  [LearnerHandler-/10.10.34.12:59480:LearnerHandler@562] - Unexpected exception causing shutdown while sock still open
1  [LearnerHandler-/10.10.34.12:59455:LearnerHandler@562] - Unexpected exception causing shutdown while sock still open
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ ls
backup.sh  d1tuser.sh  greater.sh  hello.sh  items.sh  linux.sh  log.sh  num.sh  processed_logs  shell.sh  summary_report_2024-07-18.txt  task.sh  test.sh  user.sh  user1.sh  welcome.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

Day 11Task 1: Checking Exit Status

- Write a script that attempts to create a directory and checks if the command was successful. If not, print an error message.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ ls
backup.sh      dltuser.sh  hello.sh  linux.sh  num.sh          shell.sh                        task.sh  user.sh   welcome.sh
create_dir.sh  greater.sh  items.sh  log.sh    processed_logs  summary_report_2024-07-18.txt  test.sh  user1.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat create_dir.sh
mkdir /tmp/mydir
if [ $? -ne 0 ]; then
            echo "Failed to create directory /tmp/mydir"
fi
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash create_dir.sh
mkdir: cannot create directory '/tmp/mydir': File exists
Failed to create directory /tmp/mydir
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cd /tmp/
ubuntu@ip-172-31-37-26:/tmp$ ls
mydir
snap-private-tmp
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-ModemManager.service-NHvGMK
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-chrony.service-yI8lz9
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-polkit.service-wbqEL7
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-systemd-logind.service-i4Y2qT
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-systemd-resolved.service-rRfge4
ubuntu@ip-172-31-37-26:/tmp$ rm mydir/
rm: cannot remove 'mydir/': Is a directory
ubuntu@ip-172-31-37-26:/tmp$ rm -rf mydir/
ubuntu@ip-172-31-37-26:/tmp$ cd /home/ubuntu/shell-scripting/
ubuntu@ip-172-31-37-26:~/shell-scripting$ cd scripts/
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash create_dir.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cd /tmp/
ubuntu@ip-172-31-37-26:/tmp$ ls
mydir
snap-private-tmp
systemd-private-8cf71a5ab6a4459cad5608491f8cab1d-ModemManager.service-NHvGMK
```

Activate Windows
Go to Settings to activate Windows.

Task 2: Task 2: Using `if` Statements for Error Checking

- Modify the script from Task 1 to include more commands (e.g., creating a file inside the directory) and use `if` statements to handle errors at each step.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat if_statement.sh
mkdir /tmp/mydir
if [ $? -ne 0 ]; then
            echo "Failed to create directory /tmp/mydir"
                exit 1
fi

touch /tmp/mydir/myfile.txt
if [ $? -ne 0 ]; then
            echo "Failed to create file /tmp/mydir/myfile.txt"
                exit 1
fi
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash if_statement.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

Task3: Using `trap` for Cleanup

- Write a script that creates a temporary file and sets a `trap` to delete the file if the script exits unexpectedly.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat trap.sh
temp_file="/root/task3/tempfile_$$.txt"
touch "$temp_file"
echo "Temporary file created at $temp_file"

trap 'echo Deleting temporary file; rm -f "$temp_file"' EXIT

echo "Script is running..."
sleep 10

echo "Script completed successfully."
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ sudo bash trap.sh
touch: cannot touch '/root/task3/tempfile_3625.txt': No such file or directory
Temporary file created at /root/task3/tempfile_3625.txt
Script is running...
Script completed successfully.
Deleting temporary file
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

Task 4: Redirecting Errors

- Write a script that tries to read a non-existent file and redirects the error message to a file called `error.log`.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ vi redirect.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat redirect.sh
cat non_existent_file.txt 2> error.log
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash redirect.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ ls
backup.sh      error.log   if_statement.sh  log.sh          redirect.sh                    task.sh  user.sh
create_dir.sh  greater.sh  items.sh         num.sh          shell.sh                       test.sh  user1.sh
dltuser.sh     hello.sh    linux.sh         processed_logs  summary_report_2024-07-18.txt  trap.sh  welcome.sh
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat error.log
cat: non_existent_file.txt: No such file or directory
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```

Task 5: Creating Custom Error Messages

- Modify one of the previous scripts to include custom error messages that provide more context about what went wrong.

```
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ cat custom_error.sh
mkdir /tmp/mydir
if [ $? -ne 0 ]; then
            echo "Error: Directory /tmp/mydir could not be created. Check if you have the necessary permissions."
                exit 1
fi

touch /tmp/mydir/myfile.txt
if [ $? -ne 0 ]; then
            echo "Error: File /tmp/mydir/myfile.txt could not be created. Ensure the directory exists and is writable."
                exit 1
fi
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$ bash custom_error.sh
mkdir: cannot create directory ‘/tmp/mydir’: File exists
Error: Directory /tmp/mydir could not be created. Check if you have the necessary permissions.
ubuntu@ip-172-31-37-26:~/shell-scripting/scripts$
```